

END-TO-END DATA PROJECT · PARTE 1

# Do CRM/ERP à ação comercial no Databricks



Pipeline medalhão · Modelo de score de contato ·  
Genie Space · App de atendimento — construídos  
com Databricks Asset Bundles e Claude Code via  
MCP.

|              |              |            |
|--------------|--------------|------------|
| <b>3.041</b> | <b>28.7k</b> | <b>293</b> |
| CLIENTES     | PEDIDOS      | SKUS       |

► Navegue pelas páginas para conhecer o projeto

## RECONHECIMENTO

# Esse projeto não acontece sozinho

Um grande obrigado à @Jornada de Dados pelo treinamento ministrado — foi divertido, desafiador e muito gratificante. Os fundamentos ensinados foram a base de tudo que você verá a seguir.

## CERTIFICADO DE CONCLUSÃO

Guilherme G Costa

Concluiu o **Imersão Databricks + Claude Code**  
**com** carga horária de 8 horas.

**IAs utilizadas no desenvolvimento — foco em economia de custo 😊**

- **Claude Code + OpenRouter/free**
- **OpenCode + MiMo-2.5v**
- **MiMo Code + MiMo-v2.5-Pro (única versão paga)**

A integração dessas ferramentas com o olhar de um profissional experiente gerou um produto final de alta qualidade — em muito menos tempo.

## O PROBLEMA

# Times comerciais decidindo no escuro

CRM e ERP viviam em silos. Ninguém sabia, de forma sistemática, quais clientes precisavam de contato agora — e por quê.



## 37.937

visitas registradas sem cruzamento  
com risco de churn ou inadimplência



## 5.980

oportunidades no funil, muitas  
paradas sem alerta ou follow-up



## 217

clientes já inativos sem nenhuma  
ação de reativação estruturada

A pergunta que guiou o projeto:

**"Quais clientes precisam de contato, e por quê?"**

# Databricks Asset Bundle

— como o projeto foi estruturado

```
projeto_databricks/  
├─ databricks.yml      # Bundle: targets dev/prod  
├─ resources/  
│   ├─ catalogo.yml    # Schemas bronze/silver/gold + Volume  
│   ├─ pipeline.job.yml # Job orquestrado (10 tasks em DAG)  
│   └─ dashboard-comercial.lvdash.json  
├─ src/  
│   ├─ raw/conferencia.py # Valida CSVs no Volume  
│   ├─ bronze/ingestao.py # 10 tabelas Delta (tudo STRING)  
│   ├─ silver/01..04.py   # Limpeza, tipos, constraints CHECK  
│   └─ gold/05..08.py     # Dims, Fato, Marts, 9 testes  
├─ scripts/criar-catalogo.sh  
└─ dados/  crm/ + erp/    # 10 CSVs de origem
```

## targets dev/prod

Sem mode: development — evita o prefixo de schema que quebra o SQL

## DAG de 10 tasks

raw → bronze → 4x silver (paralelo) → gold dims → fato → marts → testes → dashboard

## Serverless

Notebooks PySpark sem SparkContext — usa dbutils.fs para inspecionar Volumes

# Bronze → Silver → Gold

## **BRONZE** 10 tabelas Delta — dado cru 100% STRING

- inferSchema=False — sem adivinhação de tipos
- Metadados técnicos: `_ingerido_em` e `_path_origem`
- Conferência: nº de linhas da tabela == nº de linhas do CSV (raise se divergir)
- Idempotente: overwrite a cada run

## **SILVER** 10 tabelas tipadas com contratos de qualidade

- CNPJ normalizado para 14 dígitos (`lpad` + `regexp_replace`)
- Datas: `try_to_date()` com ANSI mode — nunca aborta o job
- Deduplicação por CNPJ — mantém o cadastro mais antigo
- Constraints CHECK via `ALTER TABLE ADD CONSTRAINT (idempotente)`

## **GOLD** Dimensões, Fato, Marts e 9 testes de qualidade

- 4 dimensões: `dim_cliente`, `dim_produto`, `dim_vendedor`, `dim_calendario`
- `fato_vendas` particionada por ano/mês — exclui pedidos cancelados
- 3 marts por diretoria: Comercial, Produto, Financeiro
- 9 testes com `raise_error()` — o job para se qualquer falhar

### Licao aprendida:

mode: development prefixa schemas (`dev_user_bronze`) e quebra todo o SQL.  
Solucao: `pause_status: PAUSED` no job.

# Star schema + 3 marts

prontos para BI e Genie



## 3 MARTS DE CONSUMO:

### **mart\_vendas\_ por\_vendedor**

Ticket médio,  
meta, atingimento

### **mart\_produto\_ performance**

Curva ABC,  
rank mensal

### **mart\_financeiro\_ recebimento**

Adimplência,  
atraso médio

# 9 testes automáticos + contratos na Silver



## Receita gold

= silver ( $\pm 0,01$ )  
raise\_error() se falhar



## CNPJ único

na silver  
raise\_error() se falhar



## data\_pedido

sem nulos  
raise\_error() se falhar



## Receita negativa

só com devolução  
raise\_error() se falhar



## Volume fato\_vendas

entre 140k-250k  
raise\_error() se falhar



## pedido\_id

sem órfão  
raise\_error() se falhar



## cliente\_id

sem órfão  
raise\_error() se falhar



## Mart produto

$\equiv$  fato ( $\pm 0,01$ )  
raise\_error() se falhar



## CNPJ

com 14 dígitos  
raise\_error() se falhar

## Contratos na Silver (CHECK Constraints)

silver.clientes → cnpj\_14\_digitos, data\_cadastro\_nao\_nula  
silver.pedidos → valor\_total\_positivo, data\_pedido\_existe  
silver.produtos → custo\_unitario\_positivo, preco\_tabela\_positivo

## Unity Catalog:

Lineage automático · COMMENT em tabelas e colunas · PII tag em CNPJ

# Dashboard como código, versionado no bundle

O arquivo dashboard-comercial.lvdash.json é declarado no bundle (dashboard.dashboard.yml) e republicado a cada run do job — garantindo diff, rollback e re-deploy.

## Receita Líquida

Bruto - devoluções.  
Particionado ano/mes.

## Margem %

receita - custo / receita.  
Regra vive no fato.

## Ticket Médio

Receita / pedidos.  
1 pedido = 1 ticket.

## Atingimento Meta

Receita / meta\_mensal.  
NULL se sem meta.

## Queries sem prefixo

FROM fato\_vendas — sem catalog.schema. Dashboard portátil.

## Filtros cruzados

Canal × Vendedor × Período × Categoria de produto

## Refresh automático

task dashboard\_refresh no final do DAG — dados sempre atualizados

## Versionado em Git

.lvdash.json no repositório — histórico, PR e rollback

## ENCERRAMENTO

# O que fica desse projeto

- Medalhão + Unity Catalog dão confiança ao time de negócio — dado confiável é pré-requisito de ação.
- Dashboard como código (.lvdash.json no bundle): diff, rollback e deploy em um único comando.
- 9 testes automáticos com `raise_error()` fazem o job parar antes de dado inconsistente chegar ao BI.
- Agentic engineering (Claude Code + MCP + Databricks AI Dev Kit) acelerou significativamente todo o scaffold.

## STACK

---

Databricks (DABs, Unity Catalog, Serverless, AI/BI Dashboard)  
PySpark · Delta Lake · SQL ANSI · Claude Code + MCP

## Código completo no GitHub

[github.com/baludf/projeto\\_databricks](https://github.com/baludf/projeto_databricks)

Comente se quiser trocar ideia sobre agentic engineering no Databricks ↓

Parte 2 → os dados do medalhão viram score de propensão e fila de contatos